The Nota Pipeline, Unrolled

Transport hydration + the adoption deltas, explained by carrying one document through every representation.

Contents

1 Orientation

A `.nota` document is a **program that evaluates to a document**. The pipeline has two halves separated by a hard line — the **build** (Node, at SSG time) and the **client** (the reader's browser) — and the entire architecture is organized around one question: **how does each piece of the document get across that line?**

There are exactly three answers, and they define the three tiers of content:

tier	owner	how its data crosses build ⟶ client	client cost
static	`decode`	it doesn't — fully resolved at build; only HTML crosses	0
behavior	the DOM	DOM attributes + pre-rendered HTML, read back by a small vanilla module	that module (~KBs)
island	the framework	the manifest: JSON props, `moduleRef`s, slot bytes in the page	framework + island modules

Prose, lists, headings, a table of contents, cleveref-style references — static. Tooltips, copy buttons, heading anchors — behaviors. A live figure with state — an island. A document pays only for the tiers it uses; a document using none of the last two ships **zero** JavaScript.

> **Where this document sits** `design/decode.md` on the `transport-islands` branch is the spec; this tutorial is the guided tour. It also **incorporates ten design deltas** (Δ1–Δ10) agreed on top of that branch — each appears in context in an orange box, and Section 11 collects them. Where this document and the branch spec disagree, the delta is intentional.

The pipeline at a glance:

```
BUILD (Node)
  doc.nota ──reader──▸ emitted JS module            (stage A, §3)
  Doc() at ▸=false  ──▸ inert vnode tree            (stage B, §4)
  decode: trailers → normalize → index → force      (stage C, §5)
          → struct → serialize                       (stage C–D, §5–6)
  serialize ──▸ HTML + manifest {islands, behaviors} (stage D, §6)
  integrator ──▸ islands module + client entry       (stage E, §7)

CLIENT (browser)
  parse HTML (fully readable, JS or not)
  behaviors: custom elements attach                  (stage F, §8)
  hydrateIslands: markers + manifest + modules table (stage F, §8)
  — the document module is never imported —
```

2 The running example

One document, three files. The document uses every tier: prose/list/heading (static), a definition + reference (static content + a tooltip behavior), and one island with a `moduleRef` prop, a JSON prop, and static markup children.

`doc.nota`:

```
%import { Trials } from "./trials.jsx"
%import stats from "./stats.json"

# Results
```

```
We ran *forty* trials; the @Definition[id: "sd"]{standard deviation} was low.

@for (r of ["mean", "max"]) {
  - the @r was stable
}

&sd never exceeded 12 ms.

@Trials[data: moduleRef("./stats.json", stats), unit: "ms"]{
  Drag to _zoom_.
}
```

`trials.jsx` — a **module component** (transport hydration requires islands to live in modules; see §7). Note the body signature is `(children, props)` — children first:

```
import { blockComponent, h } from "@nota-lang/runtime";
import { useState } from "react";

export const Trials = blockComponent((children, props) => {
  let [zoom, setZoom] = useState(1);
  return h("figure", { class: "trials", onDblClick: () => setZoom(1) }, [
    h("svg", { width: 400, height: 120 }, [/* bars from props.data.samples */]),
    h("figcaption", {}, [`${props.data.samples.length} trials (${props.unit}), zoom
${zoom}`]),
    h("div", { class: "notes" }, [children]),
  ]);
}, "Trials");
```

`stats.json`:

```
{ "samples": [172, 185, 179] }
```

> **Why `children` gets its own `div`** A pre-rendered slot is **exclusive** on a host element: the adapter renders a `RawHtml` child as `innerHTML` and drops any siblings. Forward `@children` onto a dedicated host (here the `div.notes`), never mixed into a child list you also populate yourself.

3 Stage A — the reader emit

The oxc reader turns `doc.nota` into an ordinary JS module. Not JSX — **hyperscript**: plain calls to `h`, `Fragment`, `decode`. The compiler shim prepends the runtime import; ambient prelude names (`Heading`, `Definition`, `Ref`, …) are injected by the integrator.

```
import { h, decode, Fragment, moduleRef } from "@nota-lang/runtime";
import { Definition, Heading, Ref } from "@nota-lang/prelude";   // integrator-injected
import { Trials } from "./trials.jsx";
import stats from "./stats.json";

export default function Doc() {
  return decode(Fragment(
    h(Heading, { rank: 1 }, ["Results"]),
    "\n", "\n",
    "We ran ", h("strong", {}, ["forty"]), " trials; the ",
    h(Definition, { id: "sd" }, ["standard deviation"]), " was low.",
    "\n", "\n",
```

```
    ["mean", "max"].map((r, _i) =>
      Fragment({ key: _i }, h("nota-ul-li", {}, ["the ", r, " was stable"]))
    ),
    "\n", "\n",
    h(Ref, { id: "sd" }, []), " never exceeded 12 ms.",
    "\n", "\n",
    h(Trials, { data: moduleRef("./stats.json", stats), unit: "ms" }, [
      "Drag to ", h("em", {}, ["zoom"]), ".",
    ])
  ));
}
```

Everything downstream is determined by six facts about this emit:

1. **Markdown-ish sugar lowers to calls.** `# Results` → `h(Heading, {rank: 1}, …)` (the ambient `Heading` slot, not a bare `h1`); `*forty*` → `h("strong", …)`; `_zoom_` → `h("em", …)`; `&sd` → `h(Ref, {id: "sd"}, [])`.
2. **List items are sentinels.** `- item` → `h("nota-ul-li", {}, […])`. No `ul` exists yet; the runtime coalesces sentinel runs later (§5.5). This is what makes lists compose: a `@for`, a template, or hand-written `h` can all contribute items to one list.
3. **Whitespace is significant and explicit.** Each interior newline is its own `"\n"` text child; a blank source line therefore appears as **two adjacent** `"\n"` children — which is the paragraph-break marker (§5.5). The reader never coalesces them; the runtime never trims.
4. **`@for` emits keyed fragments.** Each iteration wraps in `Fragment({key: _i}, …)` — the key matters only if this subtree ever renders under a framework; statically the fragment dissolves.
5. **Component tags are just values.** `h(Trials, …)` references the imported binding. Nothing here says "island" yet — that is decided by what `Trials` **is** (a `blockComponent`) when the tree is evaluated.
6. **`Doc` wraps its body in `decode(...)`.** The document self-decodes when called outside any component (and only there — see the `▸` flag, next).

4 Stage B — evaluation: one call, an inert tree

The build calls `Doc()`. Every `h` / `Fragment` / `decode` consults one bit of dynamic state — the flag `▸`, "am I executing inside a component body?" At the top of `Doc()`, `▸ = false`:

primitive	`▸ = false` (document context)	`▸ = true` (component body)
`h(t, p, …k)`	build an inert vnode `⟨t, p, k⟩`; a component tag is **recorded, not invoked**	delegate to the framework adapter (`React.createElement`, …)
`Fragment`	inert `⟨FRAG, p, k⟩`	adapter fragment
`decode(v)`	run the decode pipeline (§5–6) — returns an HTML **string**	identity
`mark` / `query`	doc-state leaves (§5.3)	pointed error — doc-state is static-document-only

So `Doc()` first builds a **plain data structure**. Writing `⟨tag, props, children⟩` for vnodes, the tree handed to `decode` is:

```
⟨FRAG, {}, [
  ⟨Heading, {rank: 1}, ["Results"]⟩,                ← plain function tag: static
TEMPLATE
  "\n", "\n",
  "We ran ", ⟨"strong", {}, ["forty"]⟩, " trials; the ",
```

```
  ⟨Definition, {id: "sd"}, ["standard deviation"]⟩,  ← template (prelude slot)
  " was low.",
  "\n", "\n",
  ⟨FRAG, {key: 0}, [ ⟨"nota-ul-li", {}, ["the ", "mean", " was stable"]⟩ ]⟩,
  ⟨FRAG, {key: 1}, [ ⟨"nota-ul-li", {}, ["the ", "max",  " was stable"]⟩ ]⟩,
  "\n", "\n",
  ⟨Ref, {id: "sd"}, []⟩, " never exceeded 12 ms.",
  "\n", "\n",
  ⟨Trials, {data: ModuleRef("./stats.json"), unit: "ms"}, [   ← isComp: BOUNDARY
(deferred)
    "Drag to ", ⟨"em", {}, ["zoom"]⟩, "."
  ]⟩
]⟩
```

Three kinds of function can sit in tag position, and the distinction drives everything:

- **Templates** — plain functions (`Heading` , `Definition` , `Ref` are prelude **slots**, i.e. plain functions consulting the component registry). Expanded **eagerly at build**, spliced into the tree (§5.2). Zero client cost.
- **Boundaries** — functions marked by `inlineComponent` / `blockComponent` (`isComp`). **Never invoked** while building the tree; they sit inert until serialization renders them as islands (§6). `Trials` is one.
- **Doc-state leaves** — `mark(kind, data)` and `query(fn)` values produced by templates; resolved mid-pipeline (§5.3–5.4).

Note what `moduleRef("./stats.json", stats)` built: a branded wrapper `{spec: "./stats.json", exp: "default", value: stats}` — the **live** value rides along for the build; only `spec` / `exp` will cross the wire (§6.2).

5 Stage C — the decode pipeline

`decode(v)` at `▸ = false` runs:

```
decode(v) = serialize( struct( force( index( normalize( trailers(v) ))))))
```

Each pass below shows its effect on the running example.

5.1 `trailers` — site-registered document suffixes

Importing `@nota-lang/prelude` registered a `"definitions"` trailer. Trailers run before anything else so their queries participate in indexing. The tree is wrapped:

```
⟨FRAG, {}, [ ⟨doc tree⟩, Query(definitionsTrailer) ]⟩
```

The trailer's query will render the tooltip **bank** — but only if definitions exist, which it cannot know until the index is built. Hence: leaf now, forced later (§5.4).

5.2 `normalize` — template expansion + fragment splicing

Every **template** tag is invoked with `{children, …props}` and its result spliced in place; transparent fragments (including the keyed `@for` wrappers — keys are dropped statically) splice their children into the parent's stream. After `normalize` :

```
⟨FRAG, {}, [
  Mark(heading, {rank: 1, text: "Results"}),          ← from Heading
  Query(q_h1),                                        ← from Heading: renders the real
<h1>
  "\n", "\n",
  "We ran ", ⟨"strong", {}, ["forty"]⟩, " trials; the ",
```

```
  Mark(definition, {key: "sd", body: […]}),           ← from Definition
  ⟨"span", {id: "def-sd", class: "nota-definition"}, ["standard deviation"]⟩,
  " was low.",
  "\n", "\n",
  ⟨"nota-ul-li", {}, ["the ", "mean", " was stable"]⟩,   ← @for fragments dissolved
  ⟨"nota-ul-li", {}, ["the ", "max",  " was stable"]⟩,
  "\n", "\n",
  Query(q_ref), " never exceeded 12 ms.",             ← from Ref
  "\n", "\n",
  ⟨Trials, {…}, ["Drag to ", ⟨"em", {}, ["zoom"]⟩, "."]⟩,  ← boundary: NOT expanded
  Query(definitionsTrailer)
]⟩
```

Two things happened and one pointedly did not: templates became marks + queries + concrete hosts; keyed fragments dissolved into the sibling stream (so the sentinels are now adjacent siblings); and the `Trials` boundary was left alone — **decode descends into a boundary's children, never into its body**.

5.3 `index` — one DFS, one `DocIndex`

A single depth-first walk collects the marks:

kind	seq (per kind)	pos (global)	data
`heading`	1	1	`{rank: 1, text: "Results"}`
`definition`	1	2	`{key: "sd", body: […]}`

This is the whole trick behind two-pass constructs (ToC, numbering, refs, footnotes, bibliographies): the tree already exists in full before anything renders, so **forward reference is a scoping problem, not a temporal one**. One evaluation; tree passes do the rest.

5.4 `force` — resolve queries, remove marks

Each `Query(fn)` is called with the frozen index and its (normalized, recursively forced) output spliced in place; marks are removed. Query output may not introduce **new** marks — that is a pointed error, not a fixpoint iteration.

- `q_h1` → `⟨"h1", {id: "results"}, ["Results"]⟩` (id slugified from content; section numbering, if configured via `secset`, would be computed here from the index).
- `q_ref` → `⟨"a", {class: "nota-def-ref", href: "#def-sd", "data-nota-def": "sd"}, ["sd"]⟩`. A real link — cleveref-style resolution happens **at build**, so a JS-disabled reader gets a working reference. The `data-nota-def` attribute is the behavior tier's hook (§8.1).
- `definitionsTrailer` → sees one definition in the index, so it (a) returns the hidden tooltip **bank**, and (b) declares the document's need for the tooltip module:

```
registerClientModule("@nota-lang/prelude/client")      ← collected per render → manifest
⟨"nota-def-tooltips", {aria-hidden: "true"}, [
  ⟨"div", {class: "nota-def-tooltip", data-def: "sd"}, ["standard deviation"]⟩
]⟩
```

> **The bank is static-tier content** The tooltip **content** is decoded, indexed, rendered HTML sitting in the page — invisible via CSS until enhanced. The only thing that ships as JS is the **attach** logic. That division — content in HTML, behavior in a small module — is the behavior tier's defining move.

5.5 `struct` — the grouping passes

`struct` recursively rebuilds sibling lists, **stopping at boundaries**. Three passes in order, gated by container kind (flow containers like `FRAG` / `div` / `section` get all three; tight containers like `p` / `li` / `span` get only lists):

1. `groupLists`. Maximal runs of same-kind sentinels coalesce, bridging whitespace-only text between items; edge whitespace survives:

```
…, "\n", "\n",
⟨"ul", {}, [ ⟨"li", {}, ["the ", "mean", " was stable"]⟩,
             ⟨"li", {}, ["the ", "max",  " was stable"]⟩ ]⟩,
"\n", "\n", …
```

2. `groupParas`. Adjacent whitespace-only children are considered jointly; any run containing a blank line (≥ 2 newlines) is a **paragraph break** — it splits runs and is consumed. Maximal inline runs wrap in `⟨"p"⟩`; block siblings (the `h1`, the `ul`, the `nota-def-tooltips` element, and `Trials` — a `blockComponent` — by its `kind`) flush the current run and pass through. An `inlineComponent` would instead **join** the run — that is the entire meaning of the inline/block distinction.

3. `groupSections`. A heading owns following siblings until the next heading of rank ≤ its own, wrapped in `⟨"section"⟩`.

After `struct`:

```
⟨FRAG, {}, [
  ⟨"section", {}, [
    ⟨"h1", {id: "results"}, ["Results"]⟩,
    ⟨"p", {}, ["We ran ", ⟨"strong", {}, ["forty"]⟩, " trials; the ",
               ⟨"span", {id: "def-sd", class: "nota-definition"}, ["standard
deviation"]⟩,
               " was low."]⟩,
    ⟨"ul", {}, [ ⟨"li", …⟩, ⟨"li", …⟩ ]⟩,
    ⟨"p", {}, [⟨"a", {class: "nota-def-ref", href: "#def-sd", "data-nota-def": "sd"},
["sd"]⟩,
               " never exceeded 12 ms."]⟩,
    ⟨Trials, {…}, [ ⟨"p", {}, ["Drag to ", ⟨"em", {}, ["zoom"]⟩, "."]⟩ ]⟩,
    ⟨"nota-def-tooltips", {…}, […]⟩
  ]⟩
]⟩
```

Note `Trials`'s **children** were decoded (block component ⇒ children decode as flow ⇒ the prose became a `p`) while its **body** still has not run. Boundaries defer bodies, never children.

6 Stage D — `serialize`: HTML, islands, the manifest

`serialize` walks the struct'd tree: text escapes, hosts stringify, fragments splice — and each **boundary** renders as an island via `island(v)`:

1. Mint the next hydration id: `"1"`.
2. Serialize the boundary's (already-struct'd) children ⟶ the **slot**: `<p>Drag to <em>zoom</em>.</p>`
3. **Transport the props** — the build-time gate deciding what may cross (§6.2): `{data: ModuleRef(…), unit: "ms"}` ⟶ `{"data": {"$nota": "module", "spec": "./stats.json", "exp": "default"}, "unit": "ms"}` — recorded in the manifest under id `1`.
4. SSR the shell: set `▸ = true`, call `adapter.renderToString(adapter.h(Trials, unwrapModuleRefs(props), raw(slot)))`. Inside, the component body runs **for real** under the framework: `useState(1)` bakes `zoom 1` into the

figcaption; `onDblClick` produces no static attribute; the raw slot becomes the `div.notes`'s innerHTML. `moduleRef`s were unwrapped, so the body saw the live `stats`.

5. Emit the slot as an inert `template` followed by the marker-wrapped shell.

6.1 The final HTML

```
<section>
  <h1 id="results">Results</h1>
  <p>We ran <strong>forty</strong> trials; the
     <span id="def-sd" class="nota-definition">standard deviation</span> was low.</p>
  <ul>
    <li>the mean was stable</li>
    <li>the max was stable</li>
  </ul>
  <p><a class="nota-def-ref" href="#def-sd" data-nota-def="sd">sd</a>
     never exceeded 12 ms.</p>
  <template data-nota-slot="1"><p>Drag to <em>zoom</em>.</p></template>
  <nota-island data-hydration-id="1">
    <figure class="trials">
      <svg width="400" height="120">…</svg>
      <figcaption>3 trials (ms), zoom 1</figcaption>
      <div class="notes"><p>Drag to <em>zoom</em>.</p></div>
    </figure>
  </nota-island>
  <nota-def-tooltips aria-hidden="true">
    <div class="nota-def-tooltip" data-def="sd">standard deviation</div>
  </nota-def-tooltips>
</section>
```

Read it as a JS-disabled browser would: everything is present and functional — heading, list, the definition in place, the reference as a working fragment link, the figure's server rendering. That is the static-readability guarantee, held **by construction**: enhancement attaches to content, it never gates it.

6.2 The manifest — the wire contract

> **Δ2** One versioned wire object. The branch returned `manifest` (islands) and `clientModules` as separate fields; they are one contract now: `{v, islands, behaviors}`. It is **load-bearing API** — props cross in it — so it carries a version and breaks once, before adoption, not after sites persist manifests.

```
{
  "v": 1,
  "islands": {
    "1": {
      "comp": "Trials",
      "props": {
        "data": { "$nota": "module", "spec": "./stats.json", "exp": "default" },
        "unit": "ms"
      }
    }
  },
  "behaviors": ["@nota-lang/prelude/client"]
}
```

What may appear under `props` is exactly the **transportable** values:

- JSON data — strings, finite numbers, booleans, `null`, arrays, plain objects;

- `moduleRef(spec, value, exp?)` — the spec/export pair crosses; the client revives the **same export** from the same module (module semantics, nothing serialized);
- markup children — never props at all; they crossed as the `template` bytes.

Everything else — functions, class instances, `Map` / `Date`, symbols, circular values, vnodes — fails **the build** with a pointed error naming island, path, and fix:

```
island "Trials": prop props.data.onSelect is a function — island props must be
transportable (JSON data, moduleRef(spec, value), or markup children). Live values
belong inside the component's own module; imported values cross as moduleRef;
markup crosses as children.
```

Why this restriction is the design The deleted alternative (replay hydration) allowed arbitrary closures in island props — and paid for it globally: the client had to re-execute the **whole document** to reconstruct them, which shipped the document plus everything it imports (KaTeX, shiki, …) for even one island, and imposed a whole-document purity/determinism requirement, dynamically guarded. Transport moves the restriction to the island boundary, where it is **local** and **checked at build time**. Document code is now an unrestricted run-once build program — `Date.now()` in `%` code is legal.

Δ8 `unwrapModuleRefs` runs at **every** non-island prop consumption site (templates and slots too), not only in island SSR. This makes reader-inserted `moduleRef` wrapping sound in all positions, so a later reader release can infer `moduleRef` from `%import` bindings and delete the authored ceremony (`data: stats` instead of `data: moduleRef("./stats.json", stats)`) without a coordinated runtime break.

7 Stage E — the build: what the integrator generates

`render(Doc)` gave `{html, manifest}`. The CLI (or any prerender integrator) now generates two small modules. Neither imports the document.

7.1 The islands module

Generated by parsing the **emit's module level** (machine-generated, single-line imports — a 25-line shape parser, no JS lexer): namespace-import each module the document imports, build the `moduleRef` revival table, and scan for named island components.

```
// Generated: islands module for doc.nota
import { isComp } from "@nota-lang/runtime";
import * as $m0 from "/site/doc/trials.jsx";
import * as $m1 from "/site/doc/stats.json";

export const modules = { "./trials.jsx": $m0, "./stats.json": $m1 };

export const components = {};
for (const ns of [$m0, $m1]) {
  for (const v of Object.values(ns)) {
    if (isComp(v) && v.compName !== undefined) {
      if (components[v.compName] !== undefined && components[v.compName] !== v) {
        throw new Error(`islands module: two distinct components share the name
"${v.compName}"`);
      }
      components[v.compName] = v;
    }
```

```
  }
}
```

Δ1 The disjointness invariant. The client island graph and the document module must be disjoint: the islands module never imports the compiled document, and the build **verifies** (via the bundler's metafile) that the document module is not in the client bundle's input closure. Consequence: `%export let C = inlineComponent(…)` inside a document is a pointed error in prerender builds —

```
nota build: island "C" resolves through the document module (doc.nota) — the client
bundle must not contain the document. Move C into its own module and %import it.
```

The branch scanned the doc module for exports (`import * as $doc`), which silently dragged the entire document graph — prelude, KaTeX, shiki, all content code — back into the client bundle, collapsing the cost staircase the design exists to build. "Islands live in sibling modules" is now **the** rule, machine-checked, not a convention.

Δ3 Names are wire identity. `manifest.islands[id].comp` is resolved by name in the components map, so a duplicate `compName` across two modules would silently hydrate the wrong component with plausible props — the worst failure class in a design built on pointed errors. Duplicates are a build error (the throw above), and the reader auto-attaches names to every `inlineComponent` / `blockComponent` binding so identity never depends on remembering the constructor's second argument.

Δ9 Specs validated at build. The SSR step dry-runs `reviveProps` for every island site against the generated table, so a typo'd `moduleRef` spec (" `./stat.json` ") is a build failure naming the island and the available specs — not a console error in a reader's browser.

7.2 The client entry

```
// Generated: client entry for doc.nota
import "@nota-lang/prelude/client";                        // manifest.behaviors[0]
import { setAdapter, hydrateIslands } from "@nota-lang/runtime";
import adapter from "@nota-lang/react/client";             // client-only entry (Δ6)
import { components, modules } from "./islands.js";

setAdapter(adapter);
hydrateIslands(components, { modules });                   // manifest ← #nota-manifest
```

Generation is gated: no islands **and** no behaviors ⟶ no entry, no bundle, zero JS. Behaviors only ⟶ the entry is the first import alone (no adapter, no framework). The CLI bundles the entry (code-splitting off) and inlines it into `index.html` next to the manifest:

```
<script type="application/json" id="nota-manifest">{"v":1,"islands":{…},"behaviors":
[…]}</script>
<script type="module">/* bundled entry */</script>
```

— which is why a built document directory still works over `file://` .

Δ6 Adapter split. `@nota-lang/react/client` exports `h` / `Fragment` / `hydrate` without importing `react-dom/server` ; the SSR half lives in `@nota-lang/react/server` . The branch shipped `renderToString` to every client (≈ 30 kB gz of dead weight) because the adapter was one module. The split also completes transport's own thesis at the package level: **the client never renders to strings**.

8 Stage F — the client

8.1 Behaviors attach

`@nota-lang/prelude/client` runs `customElements.define("nota-def-tooltips", …)`. The bank element already sitting in the HTML **upgrades**; its `connectedCallback` installs three delegated listeners (click / dblclick / Escape). Click a `[data-nota-def]` reference → clone that key's bank entry → position it with floating-ui (`computePosition` + flip/shift/offset). Disconnection (SPA route unmount, iframe teardown) removes the listeners — mount/teardown for free, from the platform's own lifecycle.

No framework, no props, no manifest reads beyond the entry generation: the DOM **is** the data — `data-nota-def="sd"` on the anchor, `data-def="sd"` in the bank.

> **Δ7 Every consumer, not just the CLI.** Custom-element registries are per-window, so the playground's preview iframe must load behavior modules **inside the frame** (a module script written during `doc.write`); embedded apps (react-router) import behavior modules once at site setup (`root.tsx`) — render-time registrations cannot drive static route imports, and site-granularity pay-for-use is the right trade there. The branch wired behaviors for the CLI only; "defs work in every use case" is an adoption gate.

8.2 Islands hydrate

`hydrateIslands(components, {modules})` — the entire client driver, no capture, no replay:

1. Read the manifest from `#nota-manifest`.
2. Walk `[data-hydration-id]` markers in document order (outer before nested, for free).
3. For marker `1`: site = `{comp: "Trials", props: {…}}`; resolve `components["Trials"]` (falling back to the `registerComponents` registry — the `--setup` override path).
4. `reviveProps`: `{"$nota": "module", "spec": "./stats.json", "exp": "default"}` → `modules["./stats.json"].default` — the **same object** the build used, by module semantics.
5. Slot: `template[data-nota-slot="1"]`'s innerHTML → `raw(slot)`.
6. `adapter.hydrate(adapter.h(Trials, props, raw(slot)), marker)` — React attaches over the server shell; `useState(1)` initializes; `onDblClick` binds. The figure is live.

Inside the component body, `▸ = true` — not because anyone set a global on the client, but because the `blockComponent` wrapper is `(props) => withFlag(true, () => body(props.children, props))`. The same mechanism serves build-SSR and client render; the flag's complete truth table:

context	`▸`	`h` means	`decode` means
`Doc()` body at build	false	inert vnode	full pipeline
template expansion (normalize/struct)	false	inert vnode	—
island body, build SSR (`island()`)	true	adapter `h`	identity
island body, client (via the CompFn wrapper)	true	adapter `h`	identity
client driver (`hydrateIslands`)	—	not used — calls the adapter directly	—

Per-island leniency throughout: one island's failure (unresolvable name, bad spec, adapter throw) logs a pointed error and skips it; the rest of the page hydrates.

8.3 What never happens

- The document module is never imported by the client. Its prose, its `%` code, its prelude imports (KaTeX, shiki) do not exist in the bundle — **by the Δ1 invariant, provably**.
- Nothing re-executes to "discover" anything: the manifest and the DOM carry the full hydration input. There is no determinism requirement on document code.

- No client `renderToString` — which is also why the old Solid nested-island restriction is gone.

9 The cost staircase

The design's economics, measured on the branch examples (gzip):

document profile	client JS	composition
no defs, no islands	**0**	—
defs, no islands (the paper case)	**≈ 6 kB**	custom element + floating-ui + css
islands (the interactive example)	**≈ 85–120 kB**	React + islands module + driver (85 with the Δ6 adapter split); no document, no KaTeX, no shiki
same page under replay hydration	**≈ 368 kB**	document + full import graph + framework + driver

Each step is opt-in and local: a `Definition` buys the second row; the first `blockComponent` buys the third — and **only** the framework + that component's own module graph, never the document's.

10 Sharp edges, honestly labeled

Nested islands don't survive parent re-renders.

> **Δ4** An island's static slot is re-injected as innerHTML whenever its parent island re-renders, which destroys a nested island's hydrated root (true under replay too; transport inherited it when it deleted the Solid restriction and blessed nesting). Documented limitation + a hydrate-time notice when a marker sits inside another marker's shell: prefer sibling islands.

Embedded integrators still ship the document — today.

> **Δ5** `NotaDoc` (react-router) renders the document in the browser for SPA navigations, so that integrator pays the document graph regardless of transport. But since `hydrateIslands` needs only **served bytes + manifest + islands glue** — never `Doc` — a fetch-prerendered route mode (load prerendered HTML + manifest, hydrate, never import the document) is now possible, and is the integrator's target mode. Enabling structural change, made now: the islands glue is generated as its **own virtual module**, not appended to the transformed document module, so "who imports `Doc`" stays a per-mode choice.

The `(children, props)` body signature. `blockComponent((children, props) => …)` — children first, unlike React's single-props convention. A destructure in first position (`({data}) => …`) silently reads `children`. A reader/type-level hint is queued; until then, this footnote is the hint.

Inline islands with block shells. An `inlineComponent` whose shell renders a block element (a `figure`) gets `p`-wrapped by `groupParas`; the browser's parser then ejects it — a lint warning at build is the planned fix. Use `blockComponent` for block shells.

11 The ten deltas, collected

Δ	one line	where
1	Client island graph ∩ document graph = ∅ — machine-checked; islands live in sibling modules; `%export let` islands are a prerender build error	§7.1
2	One versioned wire contract: `{v, islands, behaviors}`	§6.2
3	`compName` is wire identity: duplicates are build errors; reader auto-name-attach	§7.1

4	Nested-island slot-reset documented + hydrate-time notice	§10
5	Islands glue as its own module; react-router fetch-prerendered route mode as target	§10
6	Adapter server/client split — no `react-dom/server` on clients	§7.2
7	Behaviors wired for playground iframe + embedded apps, not CLI only	§8.1
8	`unwrapModuleRefs` at every prop consumption site — enables reader `moduleRef` inference later	§6.2
9	`moduleRef` specs validated at build (dry-run `reviveProps`)	§7.1
10	The spec's framing becomes this document's: three tiers, classified by how data crosses; the staircase and the Δ1 invariant as first-class sections	§1

12 Glossary

emit the JS module the reader produces from `.nota` (hyperscript calls, `decode` wrap).
vnode the inert build-time tree node `(tag, props, children)`; `FRAG` is the fragment tag.
sentinel a placeholder host tag carrying structure for `struct` (`nota-ul-li` → eventual `ul` / `li`).
template a plain function tag, expanded eagerly at build (prelude slots are templates).
boundary a vnode whose tag is an `inlineComponent` / `blockComponent` — deferred, becomes an island.
mark / query doc-state leaves — build-time index entries and reads against them.
trailer a registered document suffix (e.g. the definitions bank), appended before indexing.
slot an island's pre-rendered static children; crosses as `template[data-nota-slot]` bytes.
shell an island's server-rendered HTML inside its marker.
marker the `nota-island[data-hydration-id]` wrapper element the client hydrates over.
site one island's manifest record — `{comp, props}` under its id.
bank pre-rendered behavior content in the page (the tooltip entries).
manifest the versioned wire object `{v, islands, behaviors}` inlined as `#nota-manifest`.
islands module generated module exporting `components` (name → CompFn) and `modules` (spec → namespace) for the client driver.
behavior a client module attaching enhancement to served DOM (custom element); listed in `manifest.behaviors`.
▸ "inside a component body?" — the flag routing `h` / `Fragment` / `decode` between inert building and the framework adapter.